EngageTrace Software Requirements

Course: COSC565 Software Engineering
Project: EngageTrace - Course-Context-Aware Detection of Sustained LMS Activity Declines
Version: 1.0 - First iteration requirements

1. Customer Statement of Requirements

EngageTrace will provide instructors, academic advisors, and learning-analytics researchers with a web application for identifying sustained declines in students' LMS activity. Its Contextual Activity Persistence (CAP) method will assess whether an individual student's activity falls substantially below that student's own baseline, whether the decline is unusual relative to the course cohort in the same week, and whether it persists across consecutive weeks. Users will be able to inspect the evidence behind each flag and compare CAP with fixed-threshold, individual-history-only, and CUSUM detectors.

The first iteration will use public, pseudonymous OULAD data, a React dashboard, a Python detection service, and a local SQLite relational database. It will support CSV ingestion, weekly aggregation, configurable detection runs, student timelines, method comparisons, saved results, and exports. Instructors and advisors will use the dashboard for review; an analyst will manage imports and run settings. These are workflow roles in the local prototype, not claims of an implemented institutional authorization system.

Success means that a user can import a valid course dataset, run all four methods with recorded settings, inspect a student's timeline against course activity, and reproduce the displayed counts from stored results. Activity flags support human review; they do not establish disengagement, academic failure, or a need for a particular intervention.

Scope boundaries: Live LMS integration, institutional single sign-on, automated student contact, predictive grades, intervention management, production multi-user hosting, and modification of LMS records are outside this iteration. Supabase/PostgreSQL is a future hosted option, not a second database required for the initial build.

Source and assumptions: These requirements translate the supplied EngageTrace project overview into a proposed implementation. The overview's historical research results are not acceptance guarantees. CAP uses log(1 + weekly clicks), deviation from a lagged historical median, the contemporaneous cohort percentile of that deviation, and persistence across consecutive weeks. Exact baseline-window endpoints, eligibility rules, default settings, and CUSUM settings must be reconciled with the research implementation before detector acceptance. Browser-triggered runs and an analyst-only upload endpoint are proposed class-project enhancements to the research overview's offline pipeline, not claims that those features already exist.

2. Requirements Specification

2.1 Functional requirements

“Must” identifies the first-iteration acceptance scope. “Should” identifies the next priority if capacity permits. Each requirement has a corresponding user story in the separate Agile backlog.

FR-01 - Import weekly LMS activity

An analyst shall import OULAD CSV files through a Python command or an analyst-only API upload. Supported inputs are courses.csv, studentRegistration.csv, and studentVle.csv; studentInfo.csv is optional and is not needed for detection.

FR-02 - Validate and aggregate data

The system shall validate identifiers, course-presentation relationships, numeric dates and nonnegative click counts; report rejected rows; prevent duplicate imports; and aggregate accepted clicks by student and course-relative week. A documented week rule shall use week = floor(date / 7) for dates on or after course start. Pre-course events shall be reported and excluded from iteration-one detection.

FR-03 - Select a course and analysis window

The dashboard shall allow selection of dataset version, course presentation, and week range, and show the available student and week counts. Only data within the selected scope shall be analyzed.

FR-04 - Configure detector parameters

The dashboard shall expose CAP history-window, lag, log-deviation threshold, cohort-percentile threshold, and consecutive-week parameters plus settings for each baseline method. The backend shall validate settings. Defaults, formulas, eligible baseline weeks, and boundary behavior shall be documented and versioned with the engine.

FR-05 - Run CAP and three comparison detectors

The Python engine shall execute CAP, fixed threshold, individual-history-only, and CUSUM for the selected scope. CAP shall flag a student-week only when the log-scale individual deviation exceeds its threshold, its contemporaneous cohort percentile meets its threshold, and both hold for the required consecutive weeks. The chart's raw course median is a visual reference, not the CAP context statistic. Computation shall use information available through that week only. Ineligible observations shall be marked separately from unflagged observations.

FR-06 - Preserve detection runs

The system shall persist every completed run with dataset version, selected scope, engine version, all parameter values, timestamp, and each method's student-week outputs. A failed run shall have a failure status and shall not appear as completed.

FR-07 - Display student activity timelines

The dashboard shall display a selected student's weekly click totals against the same course's weekly cohort median, with detection markers and explanations of the applicable conditions. Zero activity, missing observations, and insufficient history shall be distinguishable.

FR-08 - Compare method outputs

The dashboard shall compare the four methods using flagged counts, eligible student-week denominators, flagged percentages, and CAP flag-volume reduction relative to the fixed-threshold method. It shall label this as flag volume, not verified false-positive reduction, unless labeled ground truth is available.

FR-09 - Review sustained-decline flags

The dashboard shall provide a sortable, filterable list by student pseudonym, flagged week, method, and run, linking each result to its timeline and available condition values.

FR-10 - Export analysis results

The dashboard shall export filtered results and run metadata as CSV and JSON respectively, including dataset and engine versions, parameter values, and method labels. Exports shall use pseudonymous identifiers and escape spreadsheet formula prefixes in text cells.

FR-11 - Load an earlier run

The dashboard shall list completed runs for a dataset and allow selection of a saved run without silently recomputing or changing its settings.

2.2 Non-functional requirements

The following are proposed acceptance targets to be checked during implementation.

• **NFR-01 Usability:** A user shall be able to select a course, run documented default settings, and open a flagged student's timeline in no more than five primary actions after data is imported. Labels shall explain that the activity measure is weekly clicks.

• **NFR-02 Accessibility:** The dashboard shall target WCAG 2.2 AA: keyboard access, visible focus, labeled controls, sufficient contrast, non-color flag indicators, and text/table alternatives for charts. Manual keyboard checks and automated accessibility checks shall cover the main workflow.

• **NFR-03 Performance:** On a documented reference computer with at least 4 CPU cores and 8 GB RAM, a precomputed course view containing up to 2,000 students and 40 weeks shall load within 3 seconds, and a four-method run shall finish within 60 seconds. Record measurements and hardware; show progress during computation.

• **NFR-04 Correctness and temporal integrity:** Deterministic runs on identical inputs and versions shall return identical outputs. Tests shall cover cohort-wide quiet periods, individual sustained declines, temporary drops, zero baselines, short histories, ties, and missing weeks. Later observations shall not influence earlier detection.

• **NFR-05 Data integrity and reliability:** Foreign keys, unique keys, and transactions shall prevent orphan records, duplicate weekly observations, and partially completed result sets. Interrupted imports and runs shall expose failure states. A backup and restore check shall recover dataset and run counts.

• **NFR-06 Privacy and security:** Store only pseudonymous student identifiers needed for analysis. Do not collect names, email addresses, or institutional credentials. Bind the local service to localhost; limit uploads to CSV, validate file size with a proposed 100 MB per-file limit, use parameterized SQL, and keep database writes in Python. A hosted release requires authentication, authorization, and HTTPS before external access.

• **NFR-07 Reproducibility:** Each run shall retain input checksums, dataset version, preprocessing policy, engine version, and complete settings. The offline pipeline and API shall call the same detection module; matching test inputs shall yield matching results.

• **NFR-08 Maintainability and portability:** Separate ingestion, detection, persistence, API, and presentation modules; pin dependencies; document setup and configuration; and provide a repeatable local startup procedure. Database migrations shall preserve saved run identity.

• **NFR-09 Interpretability:** Each flag shall identify its method and supporting values. Comparisons shall report eligibility and denominators. Claims about false positives or predictive accuracy require independent labels and evaluation; fewer flags alone are insufficient.

3. Data and Storage Blueprint

3.1 Data input

Input mechanism: An analyst downloads the public OULAD CSV files and supplies them to the Python ingestion command or analyst-only upload endpoint. The first iteration will not scrape websites or connect to a live LMS. The overview identifies OULAD as licensed under CC BY 4.0; dataset attribution and license information will accompany the project.

Required input fields: courses.csv supplies code_module, code_presentation, and module_presentation_length. studentRegistration.csv supplies course-presentation membership, id_student, date_registration, and date_unregistration. Optional studentInfo.csv is not required for activity detection; demographic and final-result fields will not be imported. studentVle.csv supplies code_module, code_presentation, id_student, date, and sum_click. Registration dates define the declared enrollment eligibility policy, including how missing dates are handled. These expected fields will be checked against the actual input schema during ingestion.

Processing flow: Validate files and references; compute checksums; select a course presentation; establish the enrollment/week policy; aggregate accepted daily click records to weekly totals; calculate the weekly cohort median for display and the cohort distribution of log-scale deviations for CAP; run the detectors; persist results; and expose read views to React through the Python REST API.

Zero and missing data: A week with no events becomes zero only when the import is known to cover that week and the student is eligible. An incomplete source week is missing, not zero. A missing required registration file blocks import; missing registration dates within an otherwise valid file follow a documented eligibility policy recorded with the run. Cohort medians include eligible zero-activity students and exclude missing observations. The research implementation must define any stricter detector eligibility and history rules. No future week may supply a historical baseline.

Manual input: Users choose the course, week range, student, method, and thresholds through validated controls. They do not manually edit original activity records. End-of-course outcome fields are excluded from detection inputs to avoid outcome leakage.

3.2 Database and storage

Selected solution: SQLite for the first local research iteration. It provides transactions and relational constraints without a separate database server. Python alone writes to it; React accesses data through the API. Original CSV files and result exports reside in local data directories outside the public web root; the database stores their relative locations and checksums. Raw datasets, database files, and private exports will not be committed to GitHub.

Proposed relational schema:

• **DatasetVersion:** dataset_version_id (primary key), source_name, imported_at, manifest_checksum, license_note, preprocessing_version, coverage_policy.

• **SourceFile:** source_file_id (primary key), dataset_version_id (foreign key), filename, relative_path, checksum, accepted_rows, rejected_rows, import_status.

• **CoursePresentation:** course_id (primary key), dataset_version_id (foreign key), code_module, code_presentation, length_days; unique dataset/module/presentation combination.

• **Student:** student_id (primary key, source pseudonym). No names or contact details.

• **Enrollment:** enrollment_id (primary key), course_id and student_id (foreign keys), registration_day and unregistration_day (nullable), eligibility_source; unique course/student pair.

• **WeeklyActivity:** enrollment_id (foreign key), week_index, click_count (nullable for missing), observation_status; composite primary key on enrollment_id/week_index. Observed counts must be nonnegative.

• **CohortWeek:** course_id (foreign key), week_index, median_clicks, eligible_count, observed_count; composite primary key on course_id/week_index.

• **DetectionRun:** run_id (primary key), course_id (foreign key), start_week, end_week, created_at, engine_version, parameters_json, status, error_message. Its course links to the dataset version and source checksums.

• **DetectionResult:** run_id and enrollment_id (foreign keys), method, week_index, eligible, flagged, score (nullable), explanation_json; composite primary key on run_id/enrollment_id/method/week_index. Flagged results must be eligible.

Relationships: One dataset version has many source files and course presentations. Each course has many enrollments and cohort-week records. Each enrollment has many weekly activity records. Each course has many detection runs, and each run has many method-specific results. The service shall ensure each result's enrollment belongs to the run's course.

Access and lifecycle: API reads return only the selected course/run scope. Detection requests execute in Python with validated settings and transactional writes. Enable SQLite foreign-key enforcement and index course/student/week/run lookup paths. Keep a backup before reimport or schema migration; retain input versions and completed runs for the project so results remain reproducible. Deleting a dataset must be an explicit analyst operation with dependent-record handling. A later hosted version may migrate the same relational model to Supabase/PostgreSQL with access controls.